

Vulnerabilidades en la Refactorización de Código Generado por IA

Informe de un experimento controlado

Equipo de Investigación en Ingeniería de Software Empírica Julio de 2026

Nota metodológica. Este informe describe un experimento simulado con fines ilustrativos. Los participantes, los repositorios de código y todos los datos numéricos son sintéticos, generados de forma programática para mantener coherencia estadística interna (medias, desviaciones y resultados de las pruebas de hipótesis calculados a partir de un mismo conjunto de datos). El documento sigue el modelo de cinco fases de Wohlin et al. para experimentación en ingeniería de software y la plantilla Goal-Question-Metric (GQM) de Basili para la definición del objetivo.

Resumen

Este estudio compara dos métodos de refactorización de código generado por asistentes de IA: la refactorización manual y la refactorización asistida por herramientas de análisis estático (SAST) combinadas con un asistente LLM de revisión de seguridad. Veinticuatro estudiantes avanzados de ingeniería de software refactorizaron tres artefactos de código (dos en Python, uno en Java) generados previamente por GitHub Copilot, cada uno con tres vulnerabilidades sembradas correspondientes a categorías del CWE Top 25. Los resultados simulados muestran que el grupo manual dejó sin corregir, en promedio, el 70% de las vulnerabilidades sembradas y añadió nuevas vulnerabilidades con mayor frecuencia que el grupo asistido, que redujo la tasa de vulnerabilidades residuales al 21%. Una prueba U de Mann-Whitney rechaza la hipótesis nula para las tres variables de seguridad ($p < .01$ en todos los casos), aunque el grupo asistido tardó, en promedio, 35 minutos más por sesión. El hallazgo central se interpreta a la luz del sesgo de automatización: los desarrolladores que refactorizan manualmente código ya generado por IA tienden a concentrar su escrutinio en la forma del código y no en su seguridad, bajo el supuesto implícito de que la lógica subyacente ya es correcta.

Fase 1: Definición (Scoping)

1.1 Objetivo del experimento (plantilla GQM)

Componente	Definición
Analizar	El proceso de refactorización de código fuente generado por asistentes de IA
Con el propósito de	Evaluar la introducción y la persistencia de vulnerabilidades de seguridad durante ese proceso

Componente	Definición
Con respecto a	La cantidad, el tipo (CWE) y el costo temporal asociados a cada método de refactorización
Desde el punto de vista de	Equipos de desarrollo y responsables de seguridad de aplicaciones (AppSec) que integran código generado por IA en sus repositorios
En el contexto de	Desarrolladores con formación avanzada que refactorizan código Python y Java producido por GitHub Copilot, comparando un método manual frente a uno asistido por SAST y un LLM de revisión de seguridad

De este objetivo se derivan cuatro preguntas de investigación:

- **Q1.** ¿Difiere el número de vulnerabilidades que permanecen sin corregir tras la refactorización entre el método manual y el asistido?
- **Q2.** ¿Difiere el número de vulnerabilidades nuevas que cada método introduce durante el proceso?
- **Q3.** ¿Qué categorías de vulnerabilidad (CWE) resisten más a la detección, con independencia del método empleado?
- **Q4.** ¿Qué costo en tiempo implica el uso de herramientas de asistencia, y cómo se relaciona con la reducción de vulnerabilidades observada?

Estas preguntas se responden mediante cinco métricas, todas medidas a nivel de participante (agregando los tres artefactos que cada uno refactorizó):

Métrica	Sigla	Descripción
Vulnerabilidades Residuales	VR	Vulnerabilidades sembradas (de un máximo de 9) que siguen presentes al final de la sesión
Vulnerabilidades Nuevas	VN	Vulnerabilidades no sembradas originalmente, introducidas durante la refactorización
Total de Vulnerabilidades Finales	TVF	VR + VN
Tiempo Total de Refactorización	TTR	Minutos acumulados en los tres artefactos, calculados a partir del historial de commits
Distribución por CWE	n/d	Frecuencia de cada categoría CWE entre las vulnerabilidades residuales

1.2 Hipótesis

H0 (hipótesis nula). La cantidad de vulnerabilidades de seguridad presentes en el código

final (residuales y nuevas) no difiere de forma estadísticamente significativa entre el grupo que refactoriza de manera manual y el grupo que refactoriza con asistencia de herramientas SAST y un LLM.

H1 (hipótesis alternativa). La cantidad de vulnerabilidades de seguridad presentes en el código final difiere de forma estadísticamente significativa entre ambos grupos.

La prueba se plantea a dos colas, sin asumir de antemano la dirección de la diferencia, aunque la literatura sobre sesgo de automatización anticipa que el grupo manual retendrá más vulnerabilidades.

1.3 Variables

Variable independiente. Método de refactorización, con dos niveles: *Manual* y *Asistido* (SAST + LLM).

Variables dependientes. VR, VN, TVF, TTR y la distribución por categoría CWE, definidas en la tabla anterior.

Variables de control. Lenguaje del artefacto (Python/Java, contrabalanceado entre sujetos), orden de presentación de las tres tareas (contrabalanceado mediante un cuadrado latino), conocimiento previo de seguridad (medido con un cuestionario base de 10 ítems tipo OWASP antes de la asignación a grupos) y duración máxima de la sesión (180 minutos).

Fase 2: Planificación (Planning)

2.1 Selección de sujetos

Se reclutaron 28 estudiantes de los dos últimos semestres de ingeniería de software y del primer año de una maestría profesional en la materia, todos con al menos seis meses de experiencia en prácticas o pasantías de desarrollo en Python o Java. Antes de la asignación a grupos, cada participante respondió un cuestionario base de conocimientos de seguridad (10 ítems sobre el OWASP Top 10). La asignación a grupos se aleatorizó de forma estratificada por ese puntaje, para evitar que un grupo partiera con más conocimiento previo que el otro; el puntaje promedio no mostró diferencia relevante entre grupos (Manual: $M = 6.2$ sobre 10, $DE = 1.1$; Asistido: $M = 6.4$ sobre 10, $DE = 1.3$).

Cuatro participantes fueron excluidos del análisis final: tres porque su repositorio entregado no compilaba (fallos irreversibles introducidos durante la edición manual) y uno porque abandonó la sesión antes del límite de tiempo. La muestra analizada quedó en **24 participantes, 12 por grupo.**

2.2 Diseño del experimento

El diseño es experimental, aleatorizado y entre sujetos (*between-subjects*) para el factor Método de Refactorización. Cada participante refactorizó los tres artefactos de prueba en una sesión individual de laboratorio, sin comunicación con otros participantes. El orden de presentación de los tres artefactos se contrabalanceó con un cuadrado latino de tres

órdenes, repartido de forma equitativa entre los dos grupos, para controlar efectos de aprendizaje y fatiga.

Durante el reclutamiento se informó a los participantes que el estudio evaluaba "mejoras de calidad y mantenibilidad del código", sin mencionar que la variable de interés real era la seguridad. Esta omisión, revelada en un debriefing posterior a la sesión, buscó reducir el riesgo de que los sujetos ajustaran su comportamiento por saber que se medía específicamente su atención a fallos de seguridad.

2.3 Instrumentación

Objetos de prueba. Se generaron tres artefactos con GitHub Copilot a partir de prompts realistas, cada uno con exactamente tres vulnerabilidades sembradas y varios *code smells* no relacionados con seguridad (métodos largos, código duplicado, nombres poco claros), de modo que la tarea se percibiera como una refactorización general y no como una auditoría de seguridad:

Artefacto	Descripción	Vulnerabilidades sembradas (CWE)
P1 (Python)	API REST de autenticación con Flask	CWE-89 (inyección SQL en la consulta de login), CWE-798 (clave de API sin cifrar en el código fuente), CWE-327 (contraseñas con hash MD5)
P2 (Python)	Servicio de carga y descarga de archivos	CWE-22 (recorrido de directorios en la función de descarga), CWE-502 (deserialización insegura con <code>pickle</code>), CWE-798 (clave de sesión sin cifrar en el código fuente)
J1 (Java)	Endpoint de búsqueda y carga con Spring Boot	CWE-79 (XSS reflejado en el endpoint de búsqueda), CWE-502 (deserialización insegura de un objeto Java), CWE-611 (procesamiento de XML externo sin restricciones)

Herramientas del grupo asistido. Semgrep y Bandit para el análisis estático de los artefactos Python, SonarQube para el artefacto Java, y un asistente LLM de revisión de seguridad integrado como extensión de VS Code, que explicaba en lenguaje natural cada hallazgo del SAST y proponía una corrección que el participante podía aceptar, modificar o rechazar.

Condición del grupo manual. VS Code sin complementos de análisis estático ni asistente LLM. Se permitió el uso de documentación oficial de Python, Flask, Java y Spring, pero no de motores de búsqueda ni asistentes de código.

Evaluación de resultados. Dos evaluadores, sin conocer a qué grupo pertenecía cada entrega, calificaron cada repositorio final con una rúbrica basada en las nueve vulnerabilidades sembradas y en un registro abierto para vulnerabilidades nuevas no previstas. Cada calificación se contrastó con un escaneo automatizado de CodeQL, usado como referencia adicional. La concordancia entre evaluadores fue alta (kappa de Cohen = 0.87).

2.4 Amenazas a la validez

Validez interna. Dos amenazas principales: (a) el sesgo de expectativa, mitigado ocultando el propósito de seguridad del estudio bajo un marco de "calidad y mantenibilidad"; (b) el sesgo del evaluador, mitigado mediante calificación ciega por dos revisores independientes con verificación cruzada automatizada (CodeQL).

Validez externa. Dos amenazas principales: (a) el uso de estudiantes en lugar de desarrolladores profesionales, que limita la generalización directa a entornos de producción; se mitigó exigiendo experiencia previa mínima en prácticas profesionales, aunque la limitación se reconoce explícitamente en la Fase 5; (b) el conjunto reducido de tres artefactos y nueve vulnerabilidades no cubre toda la superficie de CWE relevante en un repositorio real; se mitigó seleccionando categorías del CWE Top 25 de MITRE con alta frecuencia reportada en código generado por asistentes de IA.

Validez de constructo. La operacionalización de "vulnerabilidad" se limitó a defectos verificables de forma determinista frente a un CWE reconocido, lo que excluye fallos de seguridad más ambiguos o dependientes del contexto de despliegue.

Fase 3: Ejecución (Operation)

3.1 Preparación

Cada sesión comenzó con un tutorial de 30 minutos que explicaba la mecánica del experimento (objetivo aparente de calidad y mantenibilidad, forma de entrega vía Git, convención de mensajes de commit) y una ronda de práctica con un artefacto de descarte que no formó parte del análisis. El grupo asistido recibió además 15 minutos de familiarización con el panel del SAST y el chat del asistente LLM. Se indicó a todos los participantes que hicieran commits frecuentes conforme avanzaran, lo que permitió reconstruir el tiempo de refactorización (TTR) a partir de las marcas de tiempo del repositorio sin depender de un cronómetro manual que pudiera alterar su ritmo de trabajo.

3.2 Ejecución

Los participantes trabajaron en cubículos individuales, sin acceso a internet salvo el sandbox controlado del asistente LLM para el grupo asistido (sin capacidad de búsqueda externa). Cada uno refactorizó los tres artefactos en el orden asignado por el cuadrado latino, con un límite de 180 minutos por sesión completa; quienes terminaban antes podían finalizar la sesión. Un investigador recorrió la sala de forma periódica sin intervenir, y las sesiones se grabaron en pantalla para poder auditar el proceso si surgían dudas durante la evaluación.

3.3 Validación de datos

Antes de evaluar las vulnerabilidades, cada repositorio final se sometió a una compilación y una batería mínima de pruebas de humo: `py_compile` y `pytest` para los artefactos Python,

`mvn compile` para el artefacto Java. Los repositorios que no compilaron se excluyeron del análisis, lo que explica la reducción de la muestra de 28 a 24 participantes descrita en la Fase 2. Solo el código que compilaba y pasaba las pruebas de humo se sometió después a la evaluación de seguridad por los dos revisores ciegos, de manera que la comparación entre grupos midiera vulnerabilidades en código funcional y no simplemente diferencias en la tasa de fallos de compilación.

Fase 4: Análisis e Interpretación (Analysis and Interpretation)

4.1 Estadística descriptiva

La Tabla 1 resume las cuatro variables dependientes por grupo. La Tabla 2 presenta los datos individuales de los 24 participantes.

Tabla 1. Estadística descriptiva por grupo (n = 12 por grupo)

Variable	Grupo	Media	DE	Mediana	Mín	Máx
VR (sobre 9)	Manual	6.33	1.67	6.0	4	9
VR (sobre 9)	Asistido	1.92	0.90	2.0	1	4
VN	Manual	1.75	1.60	1.5	0	6
VN	Asistido	0.33	0.65	0.0	0	2
TVF	Manual	8.08	2.02	7.5	6	11
TVF	Asistido	2.25	1.06	2.0	1	4
TTR (min)	Manual	93.83	12.47	95.0	71	109
TTR (min)	Asistido	128.92	20.21	126.5	103	178

El patrón es consistente en las tres variables de seguridad: el grupo manual dejó sin corregir, en promedio, dos terceras partes de las vulnerabilidades sembradas (6.33 de 9) y además introdujo casi el doble de vulnerabilidades nuevas que el grupo asistido (1.75 frente a 0.33). El grupo asistido, en cambio, tardó en promedio 35 minutos más por sesión (128.92 frente a 93.83 minutos), lo que sugiere que gran parte del tiempo adicional se invirtió en revisar y decidir sobre los hallazgos del SAST y las sugerencias del LLM.

Tabla 2. Matriz de datos individuales (simulados)

Participante	Grupo	VR	VN	TVF	TTR (min)
M01	Manual	7	2	9	100
M02	Manual	4	2	6	104
M03	Manual	5	1	6	105
M04	Manual	6	0	6	103
M05	Manual	8	3	11	89
M06	Manual	8	0	8	83
M07	Manual	8	2	10	71
M08	Manual	5	1	6	107
M09	Manual	6	1	7	90
M10	Manual	6	1	7	86
M11	Manual	9	2	11	109
M12	Manual	4	6	10	79
A01	Asistido	4	0	4	135
A02	Asistido	1	0	1	126
A03	Asistido	1	0	1	103
A04	Asistido	1	0	1	120
A05	Asistido	2	0	2	138
A06	Asistido	2	1	3	135
A07	Asistido	2	0	2	146
A08	Asistido	2	0	2	127
A09	Asistido	3	0	3	178
A10	Asistido	1	1	2	105
A11	Asistido	2	2	4	114
A12	Asistido	2	0	2	120

La Tabla 3 desagrega las vulnerabilidades residuales por categoría CWE, sumando las nueve posiciones sembradas a lo largo de los 12 participantes de cada grupo (las categorías CWE-798 y CWE-502 aparecen en dos artefactos distintos, de ahí que su número máximo de instancias sea 24 en lugar de 12).

Tabla 3. Persistencia de vulnerabilidades residuales por categoría CWE

CWE	Descripción	Instancias posibles	Manual: residuales (%)	Asistido: residuales (%)
CWE-502	Deserialización insegura	24	20 (83%)	8 (33%)
CWE-611	Procesamiento inseguro de XML externo	12	10 (83%)	4 (33%)
CWE-89	Inyección SQL	12	9 (75%)	1 (8%)
CWE-327	Uso de hash criptográfico débil	12	8 (67%)	2 (17%)
CWE-22	Recorrido de directorios	12	8 (67%)	2 (17%)
CWE-79	XSS reflejado	12	7 (58%)	2 (17%)
CWE-798	Credenciales embebidas en el código	24	14 (58%)	4 (17%)

Las categorías con menor visibilidad en el código fuente, deserialización insegura y procesamiento de XML externo, son las que más resisten a la detección en ambos grupos, incluso con SAST de por medio. La inyección SQL, en cambio, muestra la mayor diferencia entre grupos: es un patrón que los analizadores estáticos detectan de forma casi determinista, por lo que su tasa de persistencia cae a un 8% en el grupo asistido frente a un 75% en el grupo manual.

4.2 Reducción de datos y prueba de hipótesis

Antes de elegir la prueba estadística se evaluó el supuesto de normalidad con Shapiro-Wilk para cada variable y grupo. La prueba rechazó la normalidad en al menos una de las dos muestras de VR, VN y TVF ($p < .05$), lo cual, sumado al carácter discreto y acotado de estas variables, llevó a usar una prueba no paramétrica para las cuatro comparaciones. La Tabla 4 resume los resultados de la prueba U de Mann-Whitney a dos colas, con el tamaño del efecto expresado como correlación biserial de rangos (r), donde valores absolutos por encima de 0.5 se consideran grandes.

Tabla 4. Resultados de la prueba U de Mann-Whitney (Manual vs. Asistido)

Variable	U	p	r	Decisión
VR	143.0	< .001	0.84	Se rechaza H0
VN	119.0	.004	0.55	Se rechaza H0
TVF	144.0	< .001	0.85	Se rechaza H0
TTR	7.0	< .001	-0.77	Se rechaza H0

Las tres variables de seguridad (VR, VN, TVF) muestran diferencias significativas con tamaños de efecto grandes, todas en la misma dirección: el grupo manual acumula más vulnerabilidades residuales, más vulnerabilidades nuevas y, por lo tanto, un total final más alto. El tiempo de refactorización también difiere de forma significativa, pero en sentido inverso: el grupo asistido invirtió más minutos por sesión. Con estos resultados se rechaza H0 para las cuatro variables y se acepta H1: el método de refactorización sí se asocia con diferencias estadísticamente significativas en la cantidad de vulnerabilidades del código final.

4.3 Interpretación de los resultados

El hallazgo más llamativo no es solo que el grupo asistido dejara menos vulnerabilidades, sino que el grupo manual, trabajando sin ninguna señal externa que apuntara a los fallos, concentró su esfuerzo casi por completo en la forma del código: nombres de variables, longitud de funciones, duplicación. Las entrevistas de debriefing informales tras cada sesión (no analizadas formalmente en este experimento) sugieren una explicación consistente con la literatura sobre automatización: al saber que el código provenía de un asistente de IA, varios participantes asumieron que la lógica de negocio ya era correcta y limitaron su revisión a aspectos estéticos, un patrón descrito en la literatura de factores humanos como sesgo de automatización o complacencia hacia sistemas automatizados (Parasuraman y Manzey, 2010).

Este sesgo ayuda a explicar también la asimetría observada en la Tabla 3. La inyección SQL y las credenciales embebidas son defectos con alta visibilidad textual (una cadena SQL construida por concatenación, una clave literal en el código), pero incluso así el grupo manual dejó sin corregir entre el 58% y el 75% de esas instancias. Si defectos tan visibles se pasan por alto con tanta frecuencia, es razonable esperar que categorías más opacas, como la deserialización insegura, con tasas de persistencia del 83% en el grupo manual, resulten prácticamente invisibles a una revisión puramente humana bajo presión de tiempo.

El grupo asistido no eliminó el problema, pero sí lo redujo de forma sustancial: la herramienta SAST actuó como un punto de interrupción obligatorio en el flujo de trabajo, forzando al desarrollador a decidir explícitamente sobre cada hallazgo en lugar de asumir que el código ya estaba bien. Ese proceso de decisión explica, al menos en parte, el tiempo adicional de 35 minutos: revisar, entender y aplicar (o descartar) una alerta de seguridad no

es una operación instantánea, incluso cuando el LLM ofrece una explicación en lenguaje natural.

Fase 5: Presentación y Conclusiones (Presentation)

5.1 Hallazgos principales

El experimento responde al objetivo planteado en la Fase 1: refactorizar código generado por IA sin apoyo de herramientas de seguridad deja, en promedio, más de dos tercios de las vulnerabilidades sembradas sin corregir y añade nuevas vulnerabilidades con una frecuencia notablemente mayor que la refactorización asistida por SAST y un LLM. La diferencia es estadísticamente significativa y de magnitud grande en las tres variables de seguridad evaluadas (VR, VN y TVF), lo que respalda la hipótesis alternativa. El costo de este beneficio es medible: el grupo asistido tardó en promedio un 37% más de tiempo por sesión.

5.2 Impacto para la industria

Estos resultados, aun siendo simulados, apuntan a un riesgo concreto para equipos que integran código generado por IA en sus repositorios: tratar ese código como un borrador "casi terminado" que solo necesita pulido estilístico puede llevar a que defectos de seguridad, algunos ya presentes en la generación original y otros introducidos durante el propio proceso de limpieza, pasen a producción sin que nadie los detecte. Algunas implicaciones prácticas para la industria:

Los equipos que refactorizan o dan mantenimiento a código de origen IA deberían tratarlo con el mismo escrutinio de seguridad que aplicarían a una contribución externa desconocida, no como un artefacto ya validado. Integrar un gate de SAST obligatorio en el pipeline de CI/CD para cualquier commit que modifique código generado o refactorizado con asistencia de IA reduce el riesgo de que la confianza en el origen del código sustituya a la verificación real. Vale también la pena formar a los equipos de revisión sobre el sesgo de automatización como fenómeno concreto, no solo como una advertencia genérica, y no confundir "el código compila y pasa las pruebas" con "el código es seguro para desplegar", ya que ambas condiciones son independientes entre sí. Por último, el tiempo adicional que exige una revisión de seguridad rigurosa debe presupuestarse de forma explícita: los resultados de este experimento sugieren que la asistencia de herramientas no vuelve el proceso más rápido, sino más lento y más seguro, un intercambio que conviene planificar en lugar de asumir como gratuito.

5.3 Trabajo futuro

Una réplica con desarrolladores profesionales de distintas empresas permitiría evaluar si el patrón observado se sostiene fuera de un entorno académico. Ampliar el conjunto de artefactos y de lenguajes (JavaScript, TypeScript, Go) daría una cobertura más amplia de categorías CWE y de ecosistemas de generación de código. Un diseño longitudinal, que siga los mismos repositorios a lo largo de varios ciclos de mantenimiento, permitiría observar si

las vulnerabilidades no detectadas en una primera refactorización reaparecen o se acumulan con el tiempo. También sería relevante comparar distintos diseños de asistente LLM, por ejemplo uno que solo explica frente a uno que aplica correcciones automáticas, para entender su efecto sobre la calibración de confianza del desarrollador. Finalmente, un estudio con metodología mixta, que incluya protocolos de pensamiento en voz alta durante la refactorización manual, podría precisar en qué momento exacto del proceso cognitivo un desarrollador decide no revisar una sección del código por asumir que ya es correcta.

Limitaciones

Todos los datos presentados en este informe son sintéticos y se generaron para ilustrar, con coherencia estadística interna, el protocolo de un experimento de cinco fases. El tamaño de muestra ($n = 24$), el número de artefactos (3) y el uso de estudiantes en lugar de profesionales limitan cualquier generalización más allá de este ejercicio metodológico. Los resultados no deben citarse como evidencia empírica real sobre la seguridad del código generado por IA.

Referencias metodológicas

Basili, V. R., & Weiss, D. M. (1984). A methodology for collecting valid software engineering data. *IEEE Transactions on Software Engineering*, SE-10(6), 728-738.

Parasuraman, R., & Manzey, D. H. (2010). Complacency and bias in human use of automation: An attentional integration. *Human Factors*, 52(3), 381-410.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer.

MITRE Corporation. *Common Weakness Enumeration (CWE)*. <https://cwe.mitre.org>